6.0 Software Design Description (SDD)

Schedulr

Meeting Scheduling Application

Team Members: Chris, Donovan, Jeron

6.1 Introduction

This Software Design Description outlines how we plan to build *Schedulr*, a full-stack web application that helps groups coordinate meeting times more efficiently. Instead of the usual back-and-forth to find a time that works for everyone, Schedulr allows users to submit their availability and then identifies and ranks the best meeting times based on who can attend.

The goal is to reduce friction in scheduling by giving clear, data-driven suggestions rather than just showing overlaps. Organizers can create an event and share a link, and participants can quickly input their availability through a web interface. The system then processes the responses and presents the most optimal time options, with features like automatically suggesting the best time and supporting preferences such as required attendance.

This document explains how the system is structured, how the components interact, and how data flows through the application. It serves as a guide for development and helps keep the team aligned as we build and improve the system.

6.1.1 System Objectives

The goal of *Schedulr* is to make it easier for groups to find meeting times that work for the most people without the usual back-and-forth communication. Instead of manually comparing schedules, users can input their availability, and the system automatically identifies and ranks the best time options.

The application brings all participant availability into one place and presents clear, data-driven recommendations to the organizer. Users can quickly create an event, share a link, and have participants submit their availability through a simple interface. The system then processes this information and highlights the time slots with the highest attendance, with options to set preferences like required participants or priority users.

Schedulr aims to save time, reduce confusion, and make scheduling more efficient, so groups can focus on planning and collaboration rather than coordinating logistics.

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 Hardware Interfaces

Schedulr runs as a web application and can be accessed on any device with a modern web browser, including laptops, tablets, and smartphones. The device must support basic input methods such as a keyboard, mouse, or touchscreen, along with an internet connection. No additional hardware or external devices are required.

- **6.1.2.2 Software Interfaces**

  Schedulr relies on several software components working together.
  - Web Browser – Provides the user interface and handles all client-side interaction.
  - Front-End Framework (e.g., React) – Manages the user interface and user input dynamically.
  - Back-End Server (e.g., Node.js/Express) – Handles requests, processes scheduling logic, and manages communication between the front end and database.
  - Database (SQL or NoSQL) – Stores event data, participant availability, and scheduling results.
  - REST API – Connects the front end and back end, allowing data to be sent and retrieved securely over HTTP.
  - Development tools such as Git and modern IDEs are used throughout the development process, with potential future integrations like calendar APIs.

6.1.2.3 Human Interfaces

Users interact with Schedulr through a clean, browser-based interface. Main features include creating an event, sharing a link, submitting availability, and viewing ranked meeting times. The design focuses on simplicity, using familiar web interactions like clicking, typing, and selecting time slots. Organizers and participants both use straightforward workflows, making the system easy to navigate without prior experience.

6.2 Architectural Design

This section explains how the main components of Schedulr are organized and how they work together to support the core features of the application. The design focuses on separating responsibilities so each part of the system can be developed and updated independently without causing issues in other areas.

Schedulr is structured into a few key components. The front end handles all user interaction, allowing organizers to create events and participants to submit their availability

through a web interface. The back end is responsible for processing requests, running the scheduling logic, and managing communication with the database. The database stores event details, participant responses, and computed results. A REST API connects the front end and back end, enabling data to be sent and retrieved efficiently.

By keeping these components separate, the system remains modular and easier to manage. Each part can be improved or expanded without heavily impacting the others, which helps reduce complexity and keeps development organized across the team.

6.2.1 Major Software Components

This section describes the main components that support the functional requirements of Schedulr. Each component plays a role in helping users create events, submit availability, and determine the best meeting times. Together, these subsystems handle the core functionality of scheduling, data processing, and user interaction within the application.

6.2.1 Major Software Components
This section describes the main components that support the core functionality of Schedulr. Each part of the system is designed to handle a specific responsibility, making the application easier to build, maintain, and extend.

6.2.1.1 Authentication Module
Responsible for creating and managing user sessions. It allows users to securely access events through unique links and ensures that only authorized users can modify event data if authentication is required.

6.2.1.2 Event Module
Handles creating, storing, and managing event data. This includes event details like title, date ranges, and time slots, as well as generating unique links that participants use to submit availability.

6.2.1.3 Availability Module
Manages participant availability input. It collects, stores, and organizes time slot selections submitted by users, ensuring the data is consistent and ready for processing.

6.2.1.4 Scheduling Engine
Processes all collected availability data and ranks meeting times based on attendance. This

component is responsible for the core logic of the application, including optional features like prioritizing certain participants or enforcing minimum attendance thresholds.

6.2.1.5 API Layer

Acts as the communication bridge between the front end and back end. It handles all requests related to events, availability, and scheduling results, ensuring data is transferred securely and efficiently.

6.2.1.6 User Interface Components

The front-end pages that users interact with, including event creation, availability input, and results display. These components allow users to navigate the system, submit data, and view ranked meeting times in a clear and simple way.

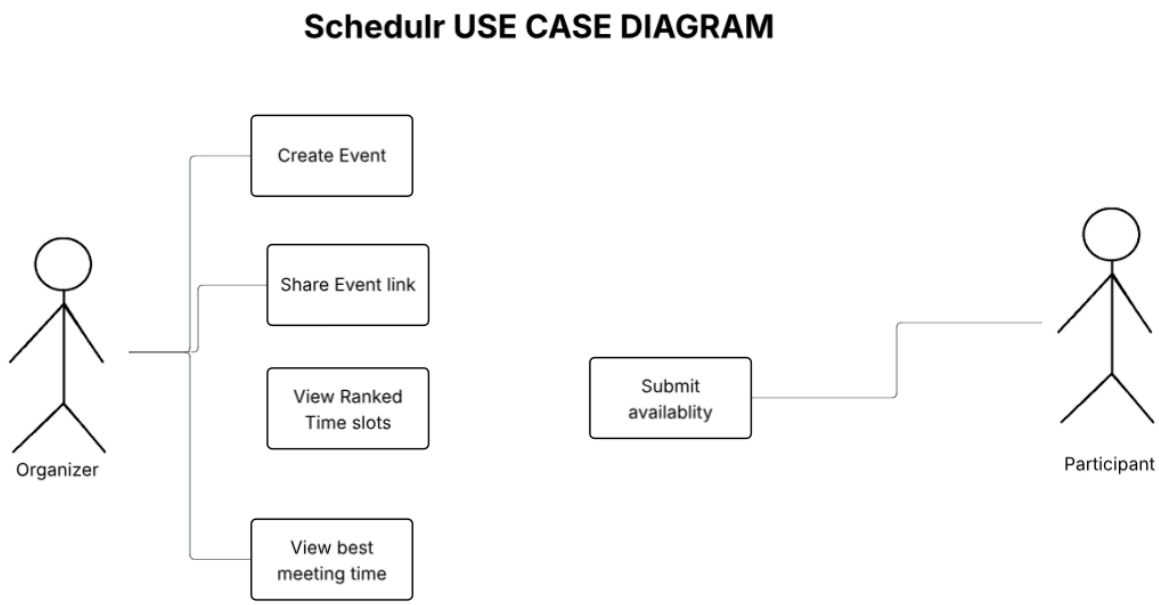
6.2.2 Major Software Interactions

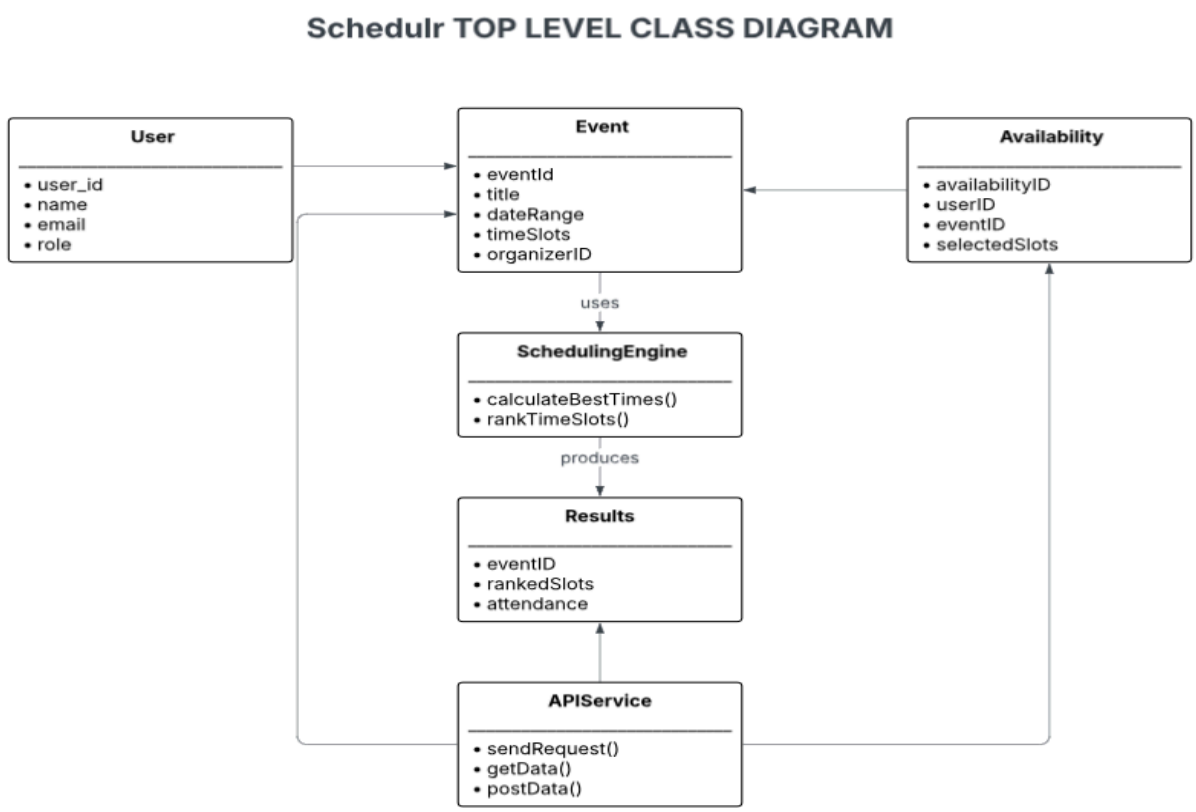
This section explains how the main components of Schedulr communicate during normal use. When an organizer creates an event, the front end sends a request through the API layer to the back end, which stores the event data in the database and generates a shareable link.

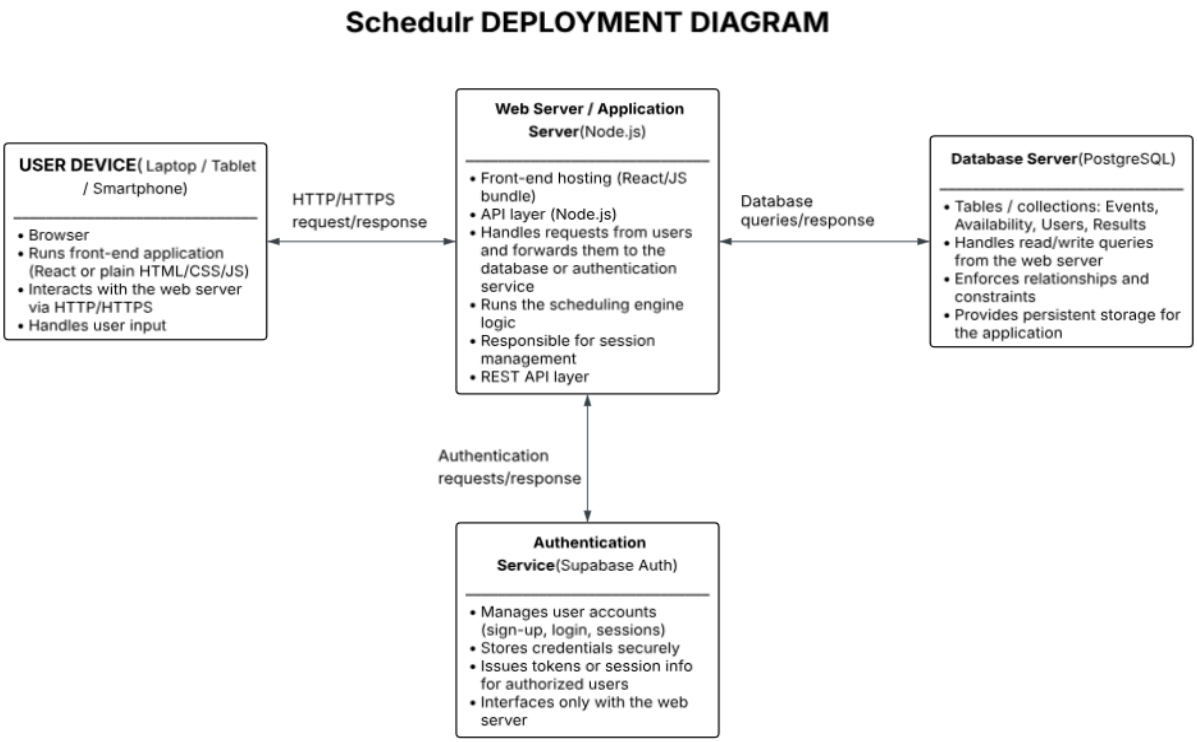
When participants access the link and submit their availability, the Availability module sends that data through the API to be stored and organized in the database. Once enough responses are collected, the Scheduling Engine retrieves the availability data, processes it, and calculates the best meeting times based on attendance.

The results are then sent back through the API to the front end, where they are displayed to the organizer in a clear, ranked format. All communication between components flows through the API layer, which keeps the system organized and ensures that data is handled consistently.

6.2.3 Architectural Design Diagrams







6.3 CSC and CSU Descriptions

Schedulr can be viewed as one main Computer Software Configuration Item (CSCI). Within this system, there are several Computer Software Components (CSCs) that handle key parts of the application, such as event creation, availability collection, and scheduling logic. Each CSC is made up of smaller Computer Software Units (CSUs), which include individual modules, functions, and UI components that implement the system's behavior.

6.3.1 Class Descriptions

This section describes the main classes and their roles within the system.

6.3.1.1 Event

This class represents a single meeting event created by a user. It stores information such as the event title, date range, time slots, and a unique identifier. The Event class is used to display event details in the UI and to store and retrieve event data from the database.

6.3.1.2 Availability

This class represents a participant's submitted availability. It stores which time slots a user is available for and is used by the system to calculate overlapping availability across all participants.

6.3.1.3 SchedulingEngine

This class handles the core scheduling logic. It processes all collected availability data and determines which time slots work best for the group. It outputs a ranked list of options based on attendance and any defined constraints.

6.3.1.4 APIService

This class manages communication between the front end and back end. It sends requests to create events, submit availability, and retrieve scheduling results, ensuring all data is properly transferred between components.

6.3.1.5 EventCreationView

A user interface component that allows organizers to create new events. It collects input such as date ranges and time slots and sends the information through the API to be stored.

6.3.1.6 ResultsView

This view displays the ranked meeting time options after availability has been processed. It presents the results in a clear format so the organizer can easily choose the best time.

6.3.2 Detailed Interface Descriptions

The API layer serves as the main interface between the user interface and the backend. All requests related to event creation, availability submission, and retrieving results are handled through this layer. The front end communicates with the back end using HTTP requests, which allows data to be sent and received securely.

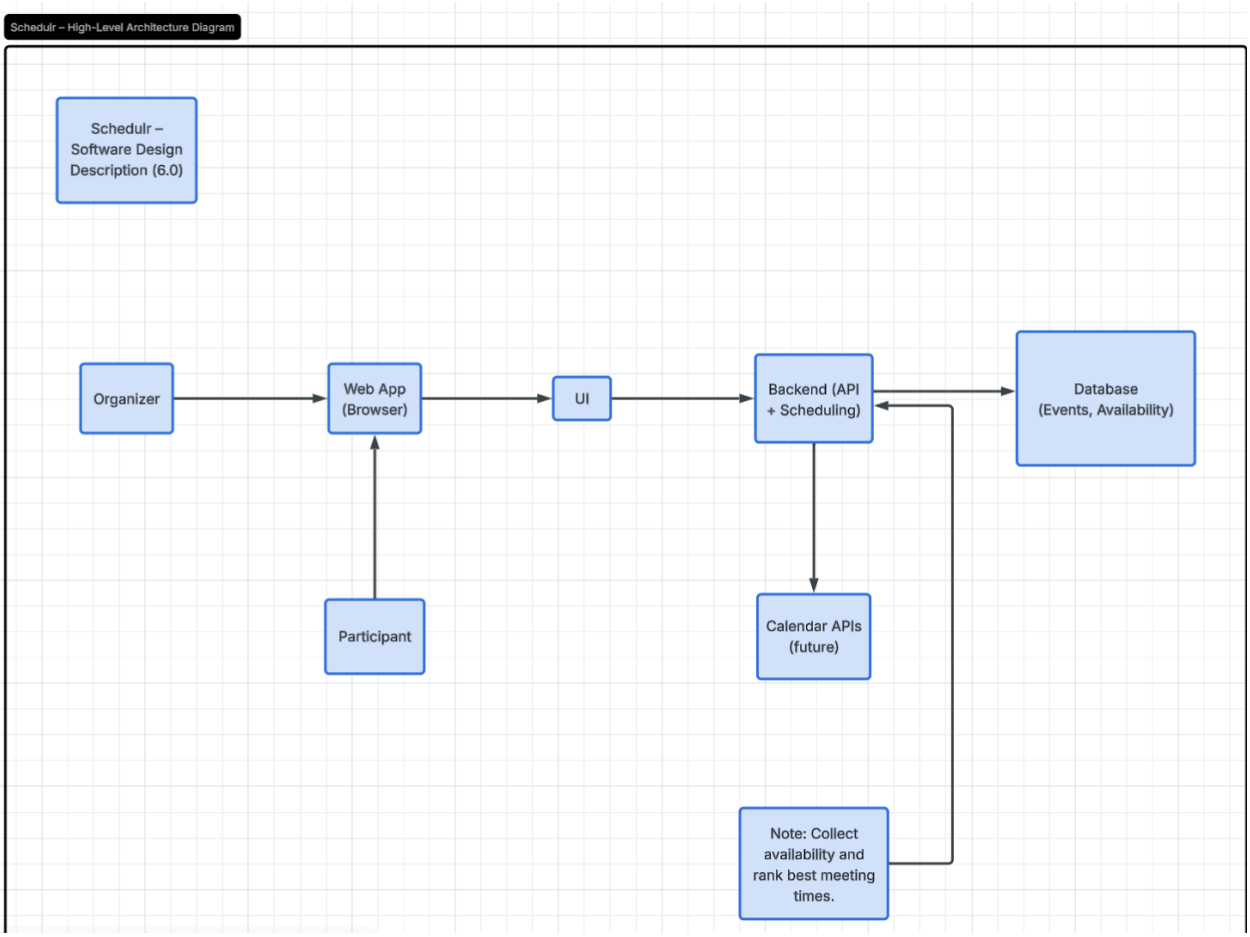
Different views pass data between each other to maintain context as users navigate the application. For example, event data is passed from the event list to the event details view so the correct information is always displayed. These interactions rely on asynchronous communication to ensure the interface remains responsive while waiting for data from the server.

6.3.3 Detailed Data Structure Descriptions

The main data structures in Schedulr are designed to support event scheduling and availability tracking. The Event structure contains key information such as the event title, date range, and available time slots. The Availability structure stores participant-selected time slots and is used to compare responses across users.

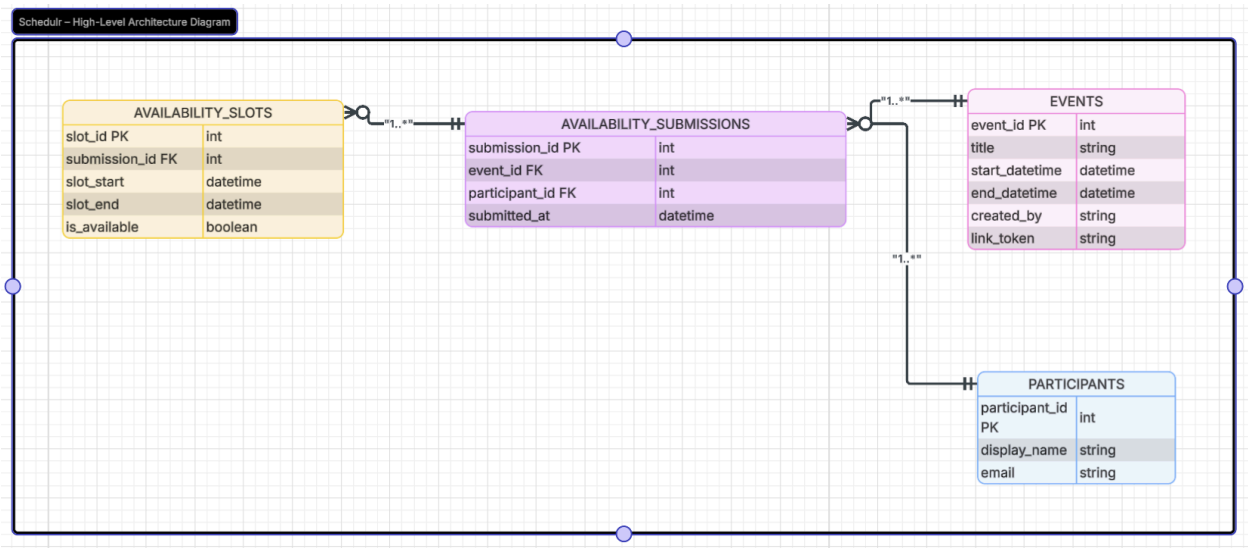
The Results structure represents the output of the scheduling process, including ranked time options based on attendance. These structures are closely tied to the database schema and ensure that data is stored and retrieved in a consistent and organized way.

6.3.4 Detailed Design Diagrams



6.4 Database Design and Description Section

6.4.1 Database Design ER Diagram Section



6.4.2 Database Access Section

Schedulr uses a database to store event data and participant availability. The main tables include an Events table, which stores event details such as title, date range, and time slots, and an Availability table, which stores the availability submissions from participants.

All database access is handled through the backend via the API layer. This layer is responsible for processing requests from the front end and performing the necessary database operations, such as creating events, submitting availability, and retrieving scheduling results. When a user interacts with the application, the front end sends a request to the backend, which then performs the appropriate database query and returns the results.

6.4.3 Database Security Section

Schedulr does not store sensitive user information beyond what is necessary for basic functionality. User authentication is handled through a secure authentication service, which manages credentials separately from the main application database.

Access to the database is restricted so that users cannot directly query or modify data without going through the proper API endpoints. This ensures that all data interactions are controlled and helps maintain the integrity and security of the system.